WEEK 04

xv6 Setup & Codebase Exploration

Problem Set

Setup xv6, read the source, write user programs.
No kernel modifications required.
All programs run entirely in user space.

Then explore the codebase like a detective.

INSIDE

PROBLEM 00

Setup & Overview.

This week is about setting up xv6 and understanding its internals by reading the source and writing simple user programs. No kernel modifications required — all programs run entirely in user space.

Setup

Clone xv6-riscv: git clone https://github.com/mit-pdos/xv6-riscv.git. Install dependencies (RISC-V toolchain + QEMU) — see theory.pdf for instructions. Build and boot: make && make qemu. Verify the shell works: ls, cat README, echo hello.

Adding programs to the build

For each problem, add a .c file to user/, add it to UPROGS in the Makefile, rebuild, and test inside QEMU.

```
# Example: adding hello_xv6
1. Save your file as user/hello_xv6.c
2. In Makefile, find UPROGS and add:  $U/_hello_xv6\
3. Rebuild:  make clean && make && make qemu
4. Run at the $ prompt:  hello_xv6
```

Suggested plan for the week

Day	What to do
1	Read theory.pdf sections 1–3. Clone xv6-riscv and get QEMU running.
2	Read theory.pdf sections 4–6. Solve Problems 1 and 2.
3	Solve Problem 3 (Fork Chain). Start Problem 4 (Codebase Detective).
4	Finish Problem 4. Write Problem 5 (Syscall Spy).
5	Review, clean up, submit.

> *Read theory.pdf before starting the problem set. Problems 1 and 4 are the minimum; Problems 3 and 4 require reading the source carefully. Problem 2 requires no code — just reading and answering questions.*

PROBLEM 01

Hello xv6!

Write a user program hello_xv6.c that prints a greeting, counts arguments, and embeds the number of system calls found in kernel/syscall.h as a #define.

What to do

Write a user program hello_xv6.c that: (1) prints a greeting that includes xv6's name and your name/alias; (2) prints the number of arguments passed to it; (3) prints each argument on its own line.

Interesting twist: print how many system calls you can find in kernel/syscall.h by embedding that count directly as a #define in your program. You'll need to read the file to count them.

Expected output

```
$ hello_xv6
Hello, xv6! This is <your_name>.
Number of arguments: 0
$ hello_xv6 hi there!
Hello, xv6! This is <your_name>.
Number of arguments: 2
arg 1: hi
arg 2: there!
xv6 knows 23 system calls.
```

Hints

- xv6 programs only use headers from inside the xv6 tree: kernel/types.h, user/user.h.
- Use argc and argv[] in main().
- Count the syscalls in kernel/syscall.h manually and #define the number.
- Every program must call exit(0) — falling off main is undefined.

What to submit

hello_xv6.c

PROBLEM 02

Codebase Detective

This problem requires no code — it is a source code exploration exercise. Open the xv6 source files in a text editor (or browse the MIT GitHub repo) and answer the following questions.

A. Process states (kernel/proc.h)

- What are the five possible states of an xv6 process?
- What state is a process in right after fork() returns in the child?
- What state is a process in between exit() and wait()?

B. The boot process (kernel/main.c)

- List the initialization steps executed in main(), in order.
- Which subsystem is initialized first? Which one last?
- At the end of main(), the kernel calls scheduler(). What does this function do? (Read kernel/proc.c.)

C. The system call table (kernel/syscall.c)

- How many system calls does xv6 currently support?
- Find the syscalls[] array. What pattern do you notice about the indices and the system call numbers in syscall.h?
- What function is called if a user program uses an invalid system call number?

D. The shell (user/sh.c)

- Find the runcmd() function. How does the shell run a command? (Look for calls to fork(), exec(), and wait().)
- What happens when you type ls | cat conceptually? Can you trace the pipe setup in the code?

What to submit

Your answers as a plain text or markdown file.

PROBLEM 03

Fork Chain

Write a user program fork_chain.c that creates a chain of N processes: process 0 → child → child → ... → Nth child. Each process prints its PID, its parent's PID, and its depth in the chain.

What to do

Take N as a command-line argument (default 5). Each process prints its PID, its parent's PID, and its depth. Only the last process (the leaf) should print "I am the leaf! No more children." Each process waits for its child to finish before exiting.

Interesting twist: after the chain completes, print the total number of processes created and the depth reached. Verify they match.

Expected output

```
$ fork_chain 3
Depth 0: PID 4, Parent PID 2
Depth 1: PID 5, Parent PID 4
Depth 2: PID 6, Parent PID 5
I am the leaf! No more children.
Total processes in chain: 3
```

Hints

- Use fork() in a loop or recursively.
- Use getpid() and the return value of fork() to distinguish parent from child.
- Every process must call exit(0).
- Use wait(0) to wait for the direct child.

What to submit

fork_chain.c

PROBLEM 04

Syscall Spy

Write a user program syscall_spy.c that calls as many different xv6 system calls as it can and prints what each one returns.

Required system calls

System call	What to expect
getpid()	Your PID
uptime()	Ticks since boot
fork()	Child PID (parent) or 0 (child)
write(1, buf, n)	Bytes written (usually n)
sleep(n)	0 on success
getpid() again	Same PID
dup(0)	New file descriptor number

Interesting twist

Use a child process that calls getpid(), then have the parent call wait() and compare the PID the parent got from fork() with the PID the child printed — they should match. Also detect the case where fork() returns 0 (you're in the child) and print "I am the child!" in that branch.

Expected output

```
$ syscall_spy
Syscall Spy Report for PID 3:
  getpid()       -> 3
  uptime()       -> 2741
  fork()         -> 4  (child PID)
  --- in child (PID 4) ---
  getpid()       -> 4
  --- back in parent (PID 3) ---
  sleep(10)      -> 0
  write(1, ...)  -> 13
  dup(0)         -> 3
  wait()         -> 4  (reaped child PID 4)
```

Hints

- Call fork() early. Inside the child (pid == 0), print your PID with a special marker, then exit(0).
- Use wait(0) in the parent to reap the child before continuing.
- write(1, "Hello, world!\n", 14) prints to stdout and returns 14.
- dup(0) duplicates stdin onto the next available fd.

What to submit

syscall_spy.c

PROBLEM 05

Submission

Keep the source files ready for the Google Form your mentor will share. The minimum submission is Problems 1 and 3; Problems 2 and 4 are optional unless your mentor asks for them.

Files to keep ready

Problem	File to keep ready
Problem 1 — Hello xv6!	hello_xv6.c
Problem 2 — Codebase Detective	answers.md or answers.txt
Problem 3 — Fork Chain	fork_chain.c
Problem 4 — Syscall Spy	syscall_spy.c

What your report should cover

Which problems you attempted. For Problem 2, your source-reading answers. For Problems 3 and 4, note anything interesting you discovered about xv6 process creation. Note one thing that surprised you and anything you got stuck on.

> *Do not submit compiled binaries — only .c source files and your answers document. If a problem has multiple files, zip them together before uploading.*

What's expected

Expectation	Notes
Clean compile	Use make inside xv6-riscv, then run in QEMU.
Correct output	Match the expected output format shown in each problem.
Honest observations	Say if something was confusing and why.

What's not expected

All four problems. A kernel modification. Deep knowledge of RISC-V assembly. Problems 1 and 3 are the

> *Stuck? Re-read theory.pdf. If something in QEMU hangs, press Ctrl-A then X to quit. If you still get stuck, message your mentor.*

minimum.